

Esercizio 2 – Server Linux

Identificazione dei server e test dei servizi TCP con Telnet

Corso di Cybersecurity
Laboratorio: VM CyberOps Workstation

Autore: team Akatsuki
7 Settembre 2026

Obiettivi del laboratorio

Questo laboratorio ha come obiettivo l'utilizzo della riga di comando Linux per identificare i server e i servizi attivi su un dato host, e l'utilizzo del client Telnet per testare direttamente il comportamento a livello applicativo di servizi TCP. Il laboratorio si articola in due parti:

- **Parte 1** — Identificazione dei server tramite `ps` e `netstat`
- **Parte 2** — Utilizzo di Telnet per testare servizi TCP (nginx sulla porta 80, SSH sulla porta 22, porta 68)

Ambiente utilizzato: VM **CyberOps Workstation**, accesso con utente `analyst`.

Parte 1 — Identificazione dei server

Passo 1 — Visualizzazione dei processi in esecuzione (`ps -elf`)

Il comando `ps -elf` mostra tutti i processi attualmente in esecuzione sul sistema. Lanciato senza privilegi elevati, l'utente può vedere PID, owner e comando di ogni processo, ma alcune informazioni sensibili sui processi altrui restano nascoste.

```
[analyst@secOps ~]$ ps -elf
F S UID          PID     PPID  C  PRI  NI ADDR  SZ  WCHAN  STIME TTY          TIME CMD
4 S root           1         0  0   80   0 - 5502 -      04:34 ?      00:00:00 /sbin/init
1 S root           2         0  0   80   0 -      0 -      04:34 ?      00:00:00 [kthreadd]
1 S root           3         2  0   80   0 -      0 -      04:34 ?      00:00:00 [pool_workqueue_release]
1 I root           4         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-rcu_gp]
1 I root           5         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-sync_wq]
1 I root           6         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-kvfree_rcu_reclaim]
1 I root           7         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-slub_flushwq]
1 I root           8         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-netns]
1 I root          10         2  0   80   0 -      0 -      04:34 ?      00:00:00 [kworker/0:1-events]
1 I root          12         2  0   80   0 -      0 -      04:34 ?      00:00:00 [kworker/u16:0-events_unbound]
1 I root          13         2  0   80   0 -      0 -      04:34 ?      00:00:00 [kworker/u16:1-ipv6_addrconf]
1 I root          14         2  0   60  -20 -      0 -      04:34 ?      00:00:00 [kworker/R-mm_percpu_wq]
1 S root          15         2  0   80   0 -      0 -      04:34 ?      00:00:00 [ksoftirqd/0]
```

Figura 1 — Output di '`ps -elf`' senza privilegi elevati

Ripetendo il comando con `sudo ps -elf`, la colonna **WCHAN** (la funzione kernel in cui il processo è in attesa) viene popolata correttamente anche per i processi di altri utenti (es. root):

```
[analyst@secOps ~]$ sudo ps -elf
[sudo] password for analyst:
F S UID          PID     PPID  C  PRI  NI ADDR  SZ  WCHAN  STIME TTY          TIME CMD
4 S root           1         0  0   80   0 - 5502 do_epo 04:34 ?      00:00:00 /sbin/init
1 S root           2         0  0   80   0 -      0 kthrea 04:34 ?      00:00:00 [kthreadd]
1 S root           3         2  0   80   0 -      0 kthrea 04:34 ?      00:00:00 [pool_workqueue_release]
1 I root           4         2  0   60  -20 -      0 rescue 04:34 ?      00:00:00 [kworker/R-rcu_gp]
1 I root           5         2  0   60  -20 -      0 rescue 04:34 ?      00:00:00 [kworker/R-sync_wq]
1 I root           6         2  0   60  -20 -      0 rescue 04:34 ?      00:00:00 [kworker/R-kvfree_rcu_reclaim]
1 I root           7         2  0   60  -20 -      0 rescue 04:34 ?      00:00:00 [kworker/R-slub_flushwq]
1 I root           8         2  0   60  -20 -      0 rescue 04:34 ?      00:00:00 [kworker/R-netns]
1 I root          10         2  0   80   0 -      0 worker 04:34 ?      00:00:00 [kworker/0:1-events]
1 I root          12         2  0   80   0 -      0 worker 04:34 ?      00:00:00 [kworker/u16:0-events_unbound]
1 I root          13         2  0   80   0 -      0 worker 04:34 ?      00:00:00 [kworker/u16:1-ipv6_addrconf]
```

Figura 2 — Output di '`sudo ps -elf`': confronto sulla colonna **WCHAN**

Domanda: perché è stato necessario eseguire `ps` come root?

Un utente non privilegiato può vedere *che* un processo esiste (PID, comando, owner), ma il kernel Linux nasconde alcuni dettagli interni sensibili — come il campo **WCHAN**, cioè la funzione kernel in cui

il processo è bloccato/in sleep — per i processi appartenenti ad altri utenti, in particolare quelli di sistema/root. Si tratta di una misura di **hardening del kernel** (restrizioni sull'accesso a /proc) che impedisce a un utente non privilegiato di raccogliere troppe informazioni sullo stato interno di processi altrui, informazioni potenzialmente sfruttabili per attacchi (es. side-channel). Con sudo, l'utente ottiene i privilegi di root e può leggere tutte le informazioni di /proc per qualsiasi processo.

Passo 2 — Albero dei processi (ps -ejH) e avvio di nginx

Dopo aver avviato nginx con privilegi elevati (sudo /usr/sbin/nginx), il comando sudo ps -ejH mostra la gerarchia dei processi in esecuzione:

```
[analyst@secOps ~]$ sudo /usr/sbin/nginx
[analyst@secOps ~]$ sudo ps -ejH
```

PID	PGID	SID	TTY	TIME	CMD
2	0	0	?	00:00:00	kthreadd
3	0	0	?	00:00:00	pool_workqueue_release
4	0	0	?	00:00:00	kworker/R-rcu_gp
5	0	0	?	00:00:00	kworker/R-sync_wq
6	0	0	?	00:00:00	kworker/R-kvfree_rcu_reclaim
7	0	0	?	00:00:00	kworker/R-slub_flushwq
8	0	0	?	00:00:00	kworker/R-netns
9	0	0	?	00:00:00	kworker/0:0-cgroup_destroy
10	0	0	?	00:00:00	kworker/0:1-events
11	0	0	?	00:00:00	kworker/0:0H-kblockd
12	0	0	?	00:00:00	kworker/u16:0-events_unbound
13	0	0	?	00:00:00	kworker/u16:1-ipv6_addrconf
14	0	0	?	00:00:00	kworker/R-mm_percpu_wq

Figura 3 — Avvio di nginx e albero dei processi con 'ps -ejH'

Domanda: come viene rappresentata la gerarchia dei processi da ps?

ps -ejH mostra i processi in forma **gerarchica ad albero**, indentando ogni processo figlio sotto il proprio processo padre. Le colonne chiave sono PID (identificativo del processo), PGID (Process Group ID), SID (Session ID) e CMD, con indentazione crescente in base al livello di discendenza. Nel nostro caso, il processo nginx master risulta padre del suo processo worker figlio, indentato di un livello sotto di lui; più in generale l'intero albero parte da systemd (PID 1) come radice di tutti i processi utente.

Passo 3 — Identificazione dei server con netstat

Il comando netstat senza opzioni mostra le connessioni di rete attive e i socket di dominio UNIX, ma non i servizi in ascolto:

```
[analyst@secOps ~]$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node  Path
unix  3      [ ]          STREAM     CONNECTED    6560
unix  3      [ ]          STREAM     CONNECTED    7774     /run/user/1000/at-spi/bus_0
unix  3      [ ]          STREAM     CONNECTED    5506
unix  3      [ ]          STREAM     CONNECTED    7648
unix  3      [ ]          STREAM     CONNECTED    7974     @/tmp/.X11-unix/X0
unix  3      [ ]          STREAM     CONNECTED    7971     @/tmp/.X11-unix/X0
unix  3      [ ]          STREAM     CONNECTED    6559
unix  3      [ ]          STREAM     CONNECTED    6563     /run/dbus/system_bus_socket
unix  3      [ ]          STREAM     CONNECTED    5509
unix  3      [ ]          STREAM     CONNECTED    5510     /run/user/1000/bus
unix  3      [ ]          DGRAM      CONNECTED    1988     /run/systemd/notify
unix  3      [ ]          STREAM     CONNECTED    6562
unix  2      [ ]          STREAM     CONNECTED    6650     /run/systemd/journal/stdout
```

Figura 4 — Output di 'netstat' senza opzioni

Per ottenere una vista utile dei server in ascolto si usa `sudo netstat -tunap`:

```
[analyst@secOps ~]$ sudo netstat -tunap
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp    0      0 0.0.0.0:80             0.0.0.0:*               LISTEN      839/nginx: master p
tcp    0      0 0.0.0.0:21             0.0.0.0:*               LISTEN      547/vsftpd
tcp    0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      425/sshd: /usr/bin/
tcp    0      0 0.0.0.0:6633           0.0.0.0:*               LISTEN      417/python3.9
tcp6   0      0 :::22                  :::*                    LISTEN      425/sshd: /usr/bin/
udp    0      0 10.0.2.15:68           0.0.0.0:*               LISTEN      330/systemd-network
```

Figura 5 — Output di 'sudo netstat -tunap'

Domanda: significato delle opzioni -t, -u, -n, -a, -p (man netstat)

Opzione	Significato
-t	Mostra le connessioni TCP
-u	Mostra le connessioni UDP
-n	Mostra indirizzi IP e porte in forma numerica (nessuna risoluzione nomi)
-a	Mostra tutte le connessioni/socket, incluse quelle in LISTEN
-p	Mostra PID e nome del programma proprietario della connessione (richiede root)

L'ordine delle opzioni è importante? No: trattandosi di opzioni short raggruppabili in stile GNU/Linux, possono essere combinate in qualsiasi ordine (es. -tunap equivale a -nputa) senza cambiare il risultato.

Domanda: protocollo, stato e PID sulla porta 80

Dall'output di `netstat -tunap`: protocollo di Livello 4 = **TCP**, stato = **LISTEN**, PID = **839** (processo `nginx: master process`). In base alla convenzione dei numeri di porta, la porta **80/TCP** è associata al protocollo **HTTP**, quindi si può ipotizzare che si tratti di un web server.

Passo 4 — Incrocio ps + grep sul PID 839

```
[analyst@secOps ~]$ sudo ps -elf | grep 839
1 S root      839      1  0  80   0 -  3723 sigsus 04:35 ?        00:00:00 nginx: master process /usr/sbin/nginx
5 S http      840     839  0  80   0 -  3827 do_epo 04:35 ?        00:00:00 nginx: worker process
0 S analyst   863     813  0  80   0 -  1615 anon_p 04:38 pts/0    00:00:00 grep 839
```

Figura 6 — Output di 'sudo ps -elf | grep 839'

Domande sull'incrocio ps + grep

Come si conclude che il PID 839 è nginx? La colonna CMD lo indica esplicitamente (nginx: master process /usr/sbin/nginx); inoltre il PID coincide con quello già individuato da netstat -tunap come proprietario della porta 80 in LISTEN — le due fonti si confermano a vicenda.

Cos'è nginx e qual è la sua funzione? Nginx è un web server open source ad alte prestazioni, usato anche come reverse proxy e load balancer. Riceve le richieste HTTP/HTTPS dai client e restituisce le risorse richieste, oppure instrada le richieste verso server backend.

Il processo 840 (utente http, padre 839): cosa significa? È il processo worker generato dal master: quest'ultimo gira come root solo per operazioni privilegiate (bind sulla porta 80), mentre i worker che gestiscono il traffico reale girano con privilegi ridotti sotto un utente dedicato (http). È un comportamento comune e voluto, adottato per limitare i danni in caso di compromissione di un worker.

Perché l'ultima riga mostra 'grep 839'? Perché il comando grep 839 stesso, mentre è in esecuzione, compare temporaneamente nell'elenco dei processi e il suo argomento ("839") fa match con il proprio pattern di ricerca — un classico effetto collaterale di ps | grep.

Parte 2 — Uso di Telnet per testare i servizi TCP

Test 1 — Telnet verso nginx (porta 80/TCP)

```
telnet 127.0.0.1 80
```

```
[analyst@secOps ~]$ telnet 127.0.0.1 80
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
ahcniwdnskd
HTTP/1.1 400 Bad Request
Server: nginx/1.28.0
Date: Mon, 07 Sep 2026 08:41:50 GMT
Content-Type: text/html
Content-Length: 157
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.0</center>
</body>
</html>
Connection closed by foreign host.
```

Figura 7 — Sessione Telnet verso la porta 80 (nginx)

Inviando una stringa casuale (ahcniwdnskd) al posto di una richiesta HTTP valida, nginx risponde con un errore **400 Bad Request**, rivelando anche la versione in uso: **nginx/1.28.0**.

Domanda: perché l'errore è stato inviato come pagina web?

Perché nginx è un server HTTP: qualunque cosa riceva, la interpreta secondo il protocollo HTTP. Non riuscendo a fare il parsing della stringa casuale come richiesta HTTP valida (che dovrebbe avere un formato tipo GET / HTTP/1.1), nginx genera un errore 400 Bad Request e lo restituisce nello stesso formato con cui risponde sempre ai client, cioè una risposta HTTP con corpo HTML.

Grazie a questo test abbiamo appreso che: (1) il processo con PID 839 è effettivamente un web server; (2) la versione di nginx è 1.28.0; (3) lo stack di rete della VM è completamente funzionante fino al Livello 7 (Applicazione) del modello OSI.

Test 2 — Telnet verso SSH (porta 22/TCP)

```
telnet 127.0.0.1 22
```

```
[analyst@secOps ~]$ telnet 127.0.0.1 22
Trying 127.0.0.1...
Connected to 127.0.0.1.
Escape character is '^]'.
SSH-2.0-OpenSSH_10.0
msaknaow
Invalid SSH identification string.
Connection closed by foreign host.
```

Figura 8 — Sessione Telnet verso la porta 22 (SSH)

A differenza di nginx, il servizio SSH invia subito un banner (SSH-2.0-OpenSSH_10.0) non appena la connessione TCP è stabilita, ancora prima che il client scriva qualcosa. Il banner serve a comunicare al client quale versione del protocollo e dell'implementazione sta usando il server, per negoziare la connessione. Inviando dati casuali (msaknaow) invece di seguire il protocollo di handshake SSH atteso, il server rileva che non si tratta di una identificazione SSH valida e restituisce l'errore **Invalid SSH identification string**, chiudendo la connessione. Questo conferma che SSH verifica il formato dei dati fin dal primo scambio, a differenza di nginx che attende una richiesta HTTP completa.

Test 3 — Telnet verso la porta 68

```
telnet 127.0.0.1 68
```

```
connection closed by foreign host.
[analyst@secOps ~]$ telnet 127.0.0.1 68
Trying 127.0.0.1...
telnet: Unable to connect to remote host: Connection refused
```

Figura 9 — Tentativo di connessione Telnet alla porta 68

Domanda: cosa succede e perché?

Il tentativo di connessione viene **rifiutato** (Connection refused). Dall'output di netstat -tunap raccolto in precedenza, il servizio in ascolto sulla porta 68 è **UDP** (non TCP), gestito da

systemd-network: la porta 68/UDP è infatti la porta standard del client **DHCP** (mentre il server DHCP usa la 67/UDP). Telnet funziona solo su TCP: non trovando alcun server TCP in ascolto su quella porta, il kernel risponde immediatamente con un pacchetto TCP RST, tradotto nel messaggio "Connection refused". Questo evidenzia un limite pratico di Telnet come strumento di analisi: può testare solo servizi basati su TCP, non su UDP.

Domande di riflessione

1. Quali sono i vantaggi dell'uso di netstat?

netstat permette di avere una visione immediata e completa delle connessioni di rete attive su un sistema: quali porte sono in ascolto (LISTEN), quali connessioni sono stabilite, quale protocollo (TCP/UDP) è in uso e — con l'opzione -p — quale processo/PID è responsabile di ciascuna connessione. Questo lo rende uno strumento fondamentale per un analista di sicurezza per: identificare rapidamente i servizi esposti su un host, individuare connessioni sospette o non autorizzate (ad esempio verso indirizzi IP esterni sconosciuti), effettuare troubleshooting di problemi di rete e, combinato con ps, correlare ogni socket di rete al processo reale che lo gestisce, verificando che corrisponda a ciò che ci si aspetta — individuando così anche eventuali processi malevoli mascherati da servizi legittimi.

2. Quali sono i vantaggi dell'uso di Telnet? È sicuro?

Telnet è utile come strumento diagnostico rapido: permette di stabilire manualmente una connessione TCP verso qualsiasi porta e osservare direttamente la risposta del servizio in ascolto (banner, versione software, comportamento del protocollo), senza bisogno di un client dedicato per ogni protocollo. È quindi prezioso per verificare rapidamente se un servizio è raggiungibile, identificarne la natura reale — non fidandosi solo del nome del processo — e raccogliere informazioni di base per attività di enumeration o troubleshooting.

Non è tuttavia sicuro come applicazione di shell remota: Telnet trasmette tutti i dati, incluse le credenziali di autenticazione, **in chiaro**, senza alcuna cifratura. Chiunque intercetti il traffico di rete (ad esempio con uno sniffer come Wireshark) può leggere in chiaro username, password e l'intero contenuto della sessione. Per questo motivo, per l'amministrazione remota sicura si utilizza SSH, che cifra l'intera comunicazione.

Conclusioni

Il laboratorio ha permesso di applicare in modo pratico gli strumenti fondamentali per l'identificazione dei server e dei servizi di rete attivi su un host Linux. Attraverso ps è stato possibile osservare i processi in esecuzione e la loro gerarchia, mentre netstat ha permesso di correlare le porte in ascolto ai processi che le gestiscono. L'utilizzo di Telnet nella Parte 2 ha inoltre dimostrato come sia possibile verificare empiricamente la natura di un servizio comunicando direttamente con esso a livello applicativo, evidenziando al contempo i limiti (solo TCP) e i rischi di sicurezza (trasmissione in chiaro) di questo strumento rispetto ad alternative moderne come SSH.